



Chapter Four

Register Transfer Language and Micro Operations

Outline

- Register transfer languages
- Bus and memory transfer
- Arithmetic and logic operations
- Shift micro operations

Basic Definitions

- ❖ A digital system is an interconnection of digital hardware modules such as registers, decoders, arithmetic elements, and control logic.
- ❖ The various modules are interconnected with common data and control paths to form a digital computer system.
- ❖ The operations executed on data stored in registers are called *microoperations*.
- ❖ A *microoperation* is an elementary operation performed on the information stored in one or more registers.

Cont..

- ❖ Examples of microoperations are shift, count, clear, and load.
- ❖ The internal hardware organization of a digital computer is best defined by specifying:
 - 1) The set of registers it contains and their function.
 - 2) The sequence of microoperations performed on the binary information stored in the registers.
 - 3) The control that initiates the sequence of microoperations.

Register Transfer Language(RTL)

- ❖ The symbolic notation used to describe the micro-operation transfer among registers is called RTL
- ❖ It is a convenient tool for describing the internal organization of digital computers
- ❖ Computer registers are designated by upper case letters (and optionally followed by digits or letters) to denote the function of the register.
 - ✓ E.g. the register that holds an address for the memory unit is usually called *a memory address register* and is designated by the name MAR.

Cont..

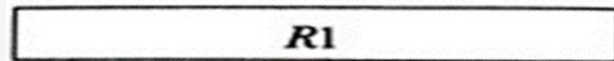
❖ Other designations for registers are

- PC (for program counter)
- IR (for instruction register)
- R1 (for processor register)

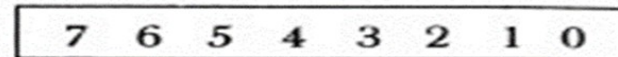
❖ The individual flip-flops in an n-bit register are numbered in sequence from 0 through n-1

❖ Figure 4-1 shows the representation of registers in block diagram form

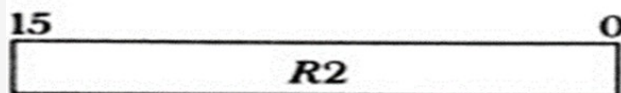
Figure 4-1 Block diagram of register.



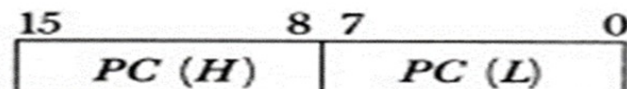
(a) Register *R*



(b) Showing individual bits



(c) Numbering of bits



(d) Divided into two parts

Register Transfer

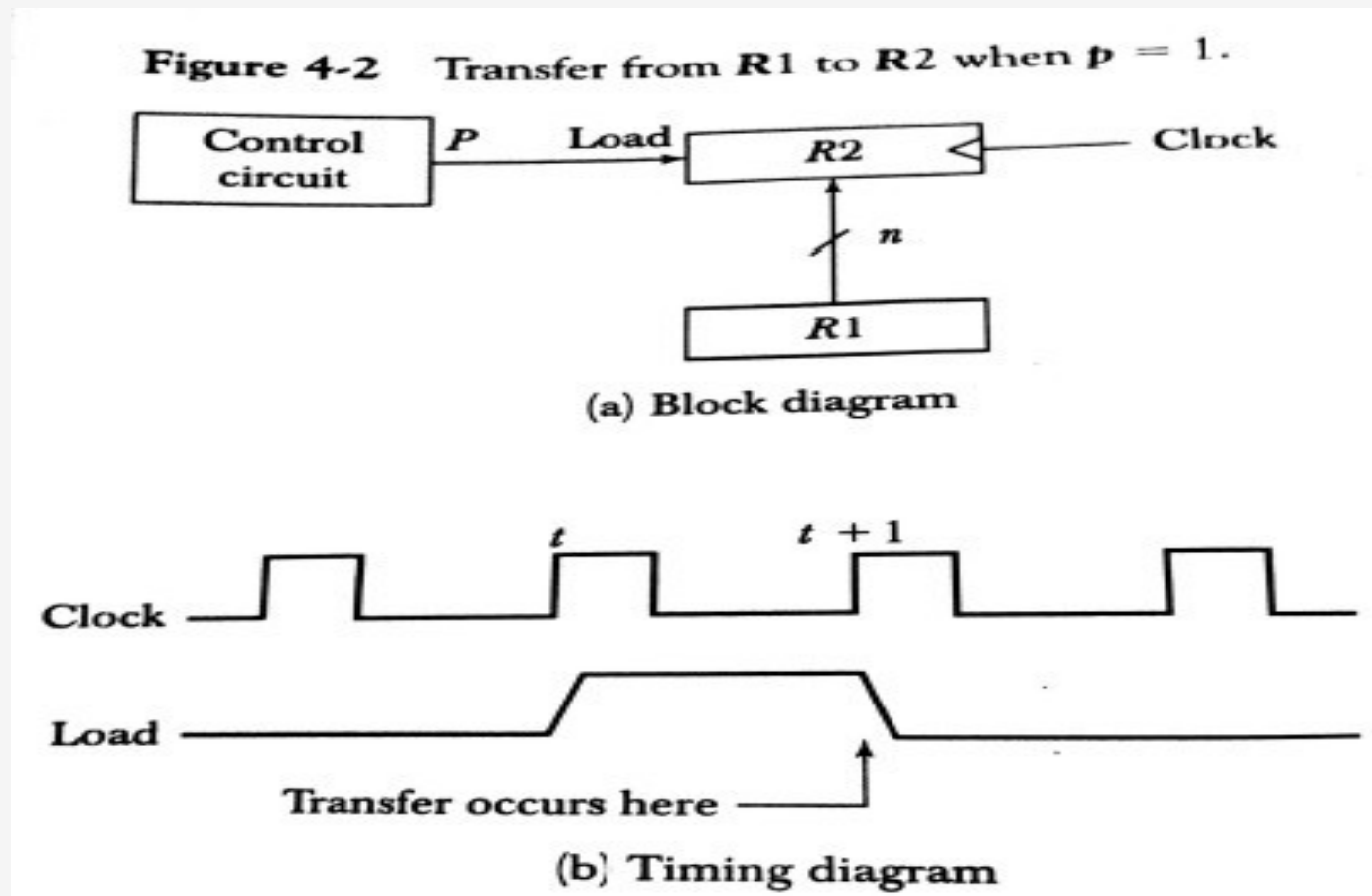
- ❖ Information transfer from one register to another is designated in symbolic form by means of *a replacement operator*.
- ❖ The statement $R2 \leftarrow R1$ denotes a transfer of the content of register $R1$ into register $R2$.
- ❖ It designates a replacement of the content of $R2$ by the content of $R1$.
- ❖ the content of the source register $R1$ does not change after the transfer.
- ❖ If we want the transfer to occur only under a predetermined control condition then it can be shown by an if-then statement.

if ($P=1$) then $R2 \leftarrow R1$

control function is included in the statement as **$P: R2 \leftarrow R1$**

Cont..

- ❖ Figure 4-2 shows the block diagram that depicts the transfer from R1 to R2.



Cont..

- ❖ The n outputs of register R1 are connected to the n inputs of register R2.
- ❖ Register R2 has a load input that is activated by the control variable P.
- ❖ It is assumed that the control variable is synchronized with the same clock as the one applied to the register.
- ❖ As shown in the timing diagram, P is activated in the control section by the rising edge of a clock pulse at time t .
- ❖ The next positive transition of the clock at time $t + 1$ finds the load input active and the data inputs of R2 are then loaded into the register in parallel.

Bus and Memory Transfers:

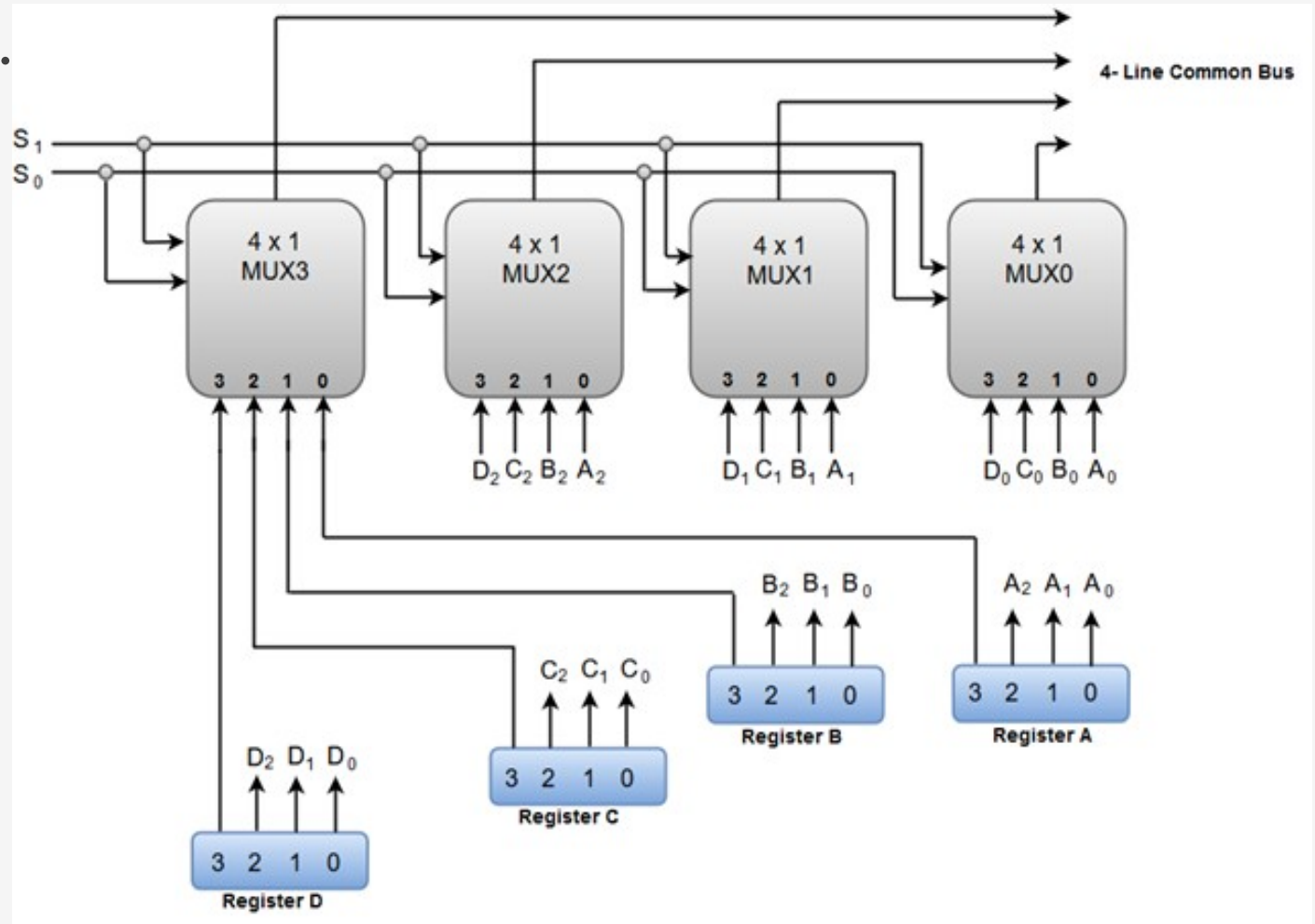
- A more efficient scheme for transferring information between registers in *a multiple-register configuration* is a **Common Bus System**.
- A common bus consists of a set of common lines, one for each bit of a register.
- **Control signals** determine which register is selected by the bus during each particular register transfer.
- Different ways of constructing a Common Bus System
 - ✓ Using Multiplexers
 - ✓ Using Tri-state Buffers

Common bus system is with multiplexers:

- The multiplexers select the source register whose binary information is then placed on the bus.

Cont..

- The construction of a bus system for four registers is shown in below .



Cont..

- ❖ The bus consists of four 4 x 1 multiplexers each having four data inputs, 0 through 3, and two selection inputs, S_1 and S_0 .
- ❖ For example, output 1 of register A is connected to input 0 of MUX 1 because this input is labelled A_1 .
- ❖ Thus MUX 0 multiplexes the four 0 bits of the registers, MUX 1 multiplexes the four 1 bits of the registers, and similarly for the other two bits.
- ❖ The two selection lines S_i and S_o are connected to the selection inputs of all four multiplexers.

Cont..

- ❖ The selection lines choose the four bits of one register and transfer them into the four-line common bus.
- ❖ When $S_1S_0 = 00$, the 0 data inputs of all four multiplexers are selected and applied to the outputs that form the bus.

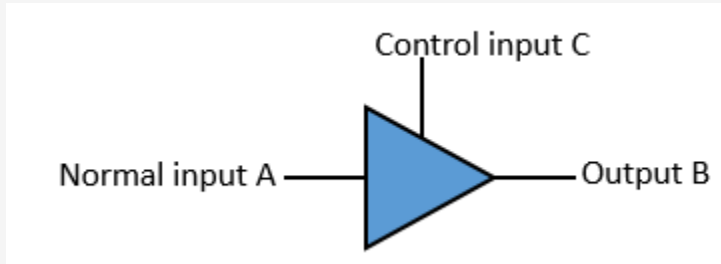
S1	S0	Register Selected
0	0	A
0	1	B
1	0	C
1	1	D

Three-State Bus Buffers:

- ❖ A bus system can be constructed with three-state gates instead of multiplexers.
- ❖ A three-state gate is a digital circuit that exhibits three states.
- ❖ Two of the states are *signals equivalent to logic 1 and 0 as in a conventional gate*.
- ❖ The third state is a *high-impedance* state.
- ❖ The high-impedance state behaves like an open circuit, which means that the output is disconnected and does not have logic significance.

Cont..

❖ The graphic symbol of a three-state buffer gate is shown in



❖ It is distinguished from a normal buffer by having both a normal *input* and a *control input*.

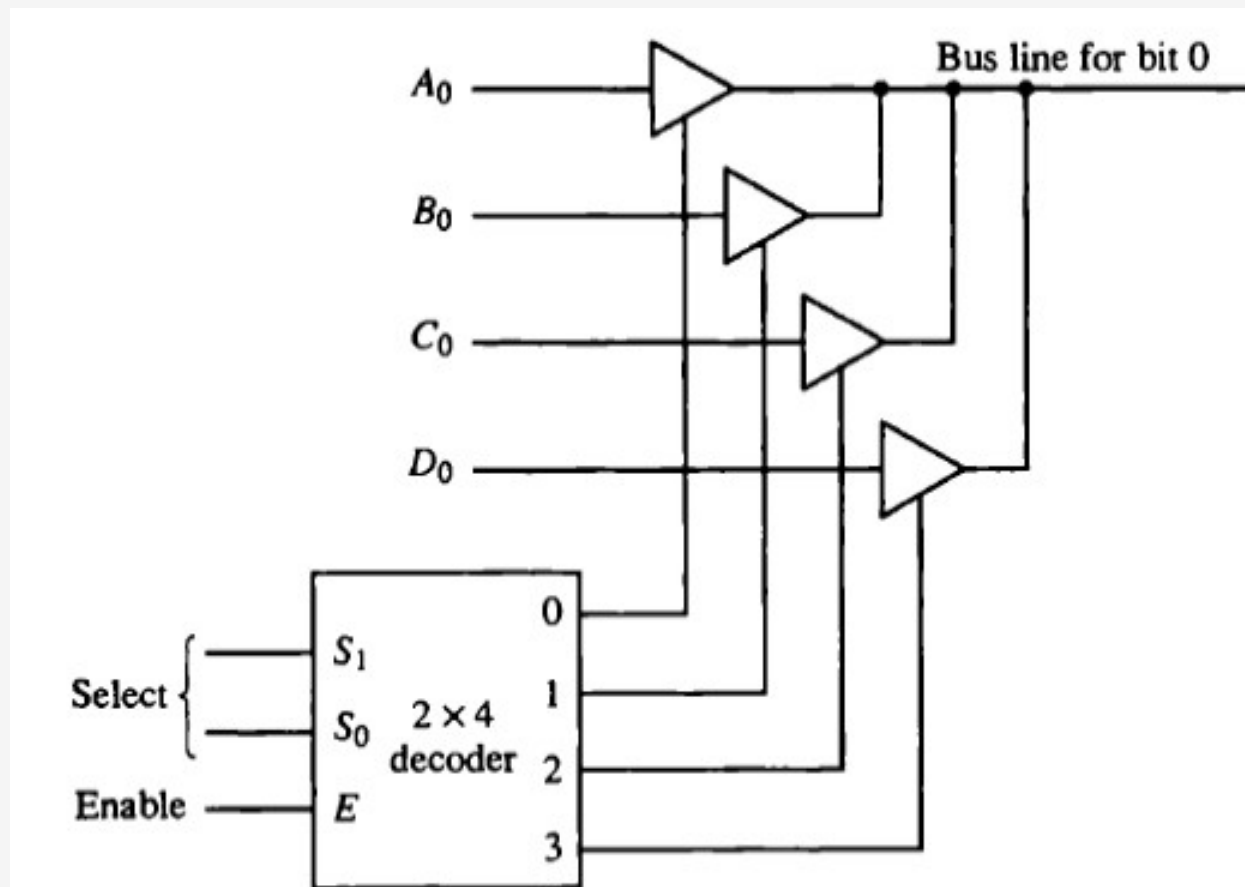
❖ The control input determines the output state.

- ✓ When the control input is equal to 1, the output is enabled
- ✓ When the control input is 0, the output is disabled .

Cont..

❖ The construction of a bus system with three-state buffers is shown in Fig.

4.5



Cont..

- ❖ The outputs of four buffers are connected together to form *a single bus line*.
- ❖ The control inputs to the buffers determine which of the four normal inputs will communicate with the bus line.
- ❖ No more than one buffer may be in the active state at any given time.
- ❖ When the **enable input** is active, one of the three-state buffers will be active, depending on *the binary value in the select inputs of the decoder*

Memory Transfer:

- ❖ The transfer of information from a memory word to the outside environment is called *a read operation*.
- ❖ The transfer of new information to be stored into the memory is called a *write operation*.
- ❖ A memory word will be symbolized by the letter M.
- ❖ The particular memory word among the many available is selected by the *memory address during the transfer*.
- ❖ This will be done by enclosing the address in square brackets following the letter M.

Cont..

- ❖ Consider a memory unit that receives the address from a register, called the address register, symbolized by **AR**.
- ❖ The data are transferred to another register, called the data register, symbolized by **DR**.
- ❖ The read operation can be stated as follows:

Read: $DR \leftarrow M[AR]$

- ❖ This causes a transfer of information into DR from the memory word M selected by the address in AR.
- ❖ The write operation transfers the content of a data register to a memory word M selected by the address.
- ❖ The write operation can be stated as follows:

Write: $M[AR] \leftarrow R_1$

Types of Micro-operations:

❖ The microoperations most often encountered in digital computers are classified into four categories:

- 1) **Register Transfer Micro-operations:** Transfer binary information from one register to another.
- 2) **Arithmetic Micro-operations:** Perform arithmetic operation on numeric data stored in registers.
- 3) **Logical Micro-operations:** Perform bit manipulation operations on data stored in registers.
- 4) **Shift Micro-operations:** Perform shift operations on data stored in registers.

Cont..

- ❖ Register Transfer Micro-operation doesn't change the information content when the binary information moves from source register to destination
- ❖ Other three types of micro-operations change the information change the information content during the transfer.

Arithmetic Micro-operations:

The basic arithmetic micro-operations are

- Addition
- Subtraction
- Increment
- Decrement
- Shift

Cont..

Add Micro-Operation:-It is defined by the following statement:

$$R_3 \leftarrow R_1 + R_2$$

- ❖ The above statement instructs the data or contents of register R1 to be added to data or content of register R2 and the sum should be transferred to register R3

Subtract Micro-Operation

- ❖ In subtract micro-operation, instead of using minus operator we take 1's compliment and add 1 to the register which gets subtracted,
- ❖ $R_3 \leftarrow R_1 + R_2' + 1$
- ❖ i.e $R_1 - R_2$ is equivalent to $R_3 \leftarrow R_1 + R_2' + 1$

Cont..

Increment/Decrement Micro-Operation

- ❖ Increment and decrement micro-operations are generally performed by adding and subtracting 1 to and from the register respectively.

$R1 \leftarrow R1 + 1$ (eg 0110, it will go to 0111 after it is incremented.)

$R1 \leftarrow R1 - 1$

Symbolic Designation	Description
$R3 \leftarrow R1 + R2$	Contents of R1+R2 transferred to R3.
$R3 \leftarrow R1 - R2$	Contents of R1-R2 transferred to R3.
$R2 \leftarrow (R2)'$	Compliment the contents of R2.
$R2 \leftarrow (R2)' + 1$	2's compliment the contents of R2.
$R3 \leftarrow R1 + (R2)' + 1$	R1 + the 2's compliment of R2 (subtraction).
$R1 \leftarrow R1 + 1$	Increment the contents of R1 by 1.
$R1 \leftarrow R1 - 1$	Decrement the contents of R1 by 1.

Binary Adder:

- Digital circuit that generates the arithmetic sum of 2 binary numbers of any lengths is called **BINARY ADDER**.
- Digital circuit that forms the arithmetic sum of 2 bits and the previous carry is called **FULL ADDER**.
- Figure 4-6 shows the interconnections of four full-adders (FA) to provide a 4-bit binary adder.

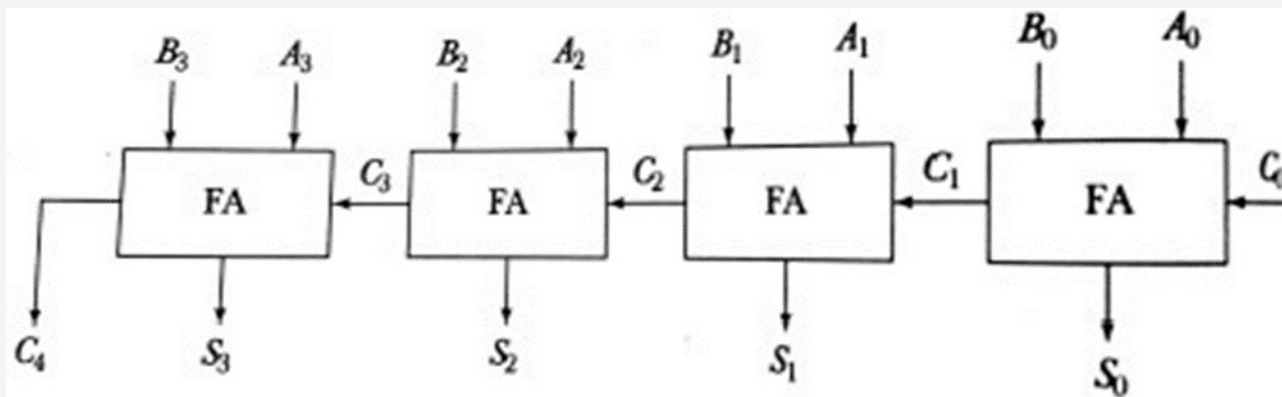


Figure 4-6 4-bit binary adder.

Cont..

- ❖ The addition and subtraction operations can be combined into one common circuit by including *an exclusive-OR gate with each full-adder*.
- ❖ A 4-bit adder-subtractor circuit figure
- ❖ When mode input $M = 0$ the circuit is an **adder** and when $M = 1$ the circuit becomes **a subtractor**.

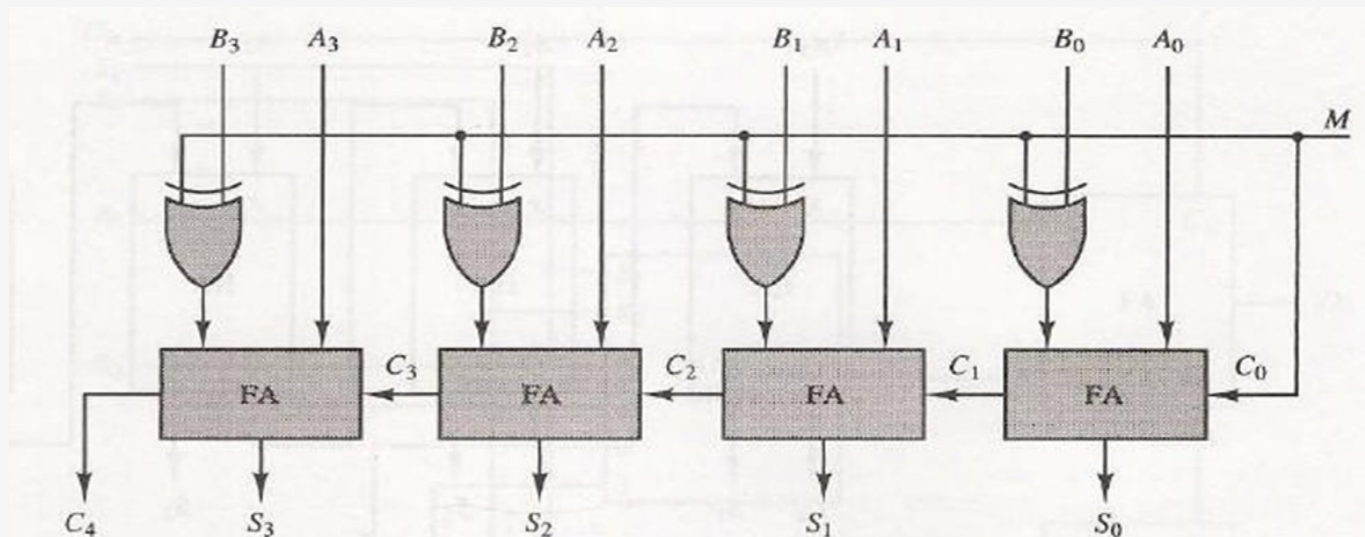
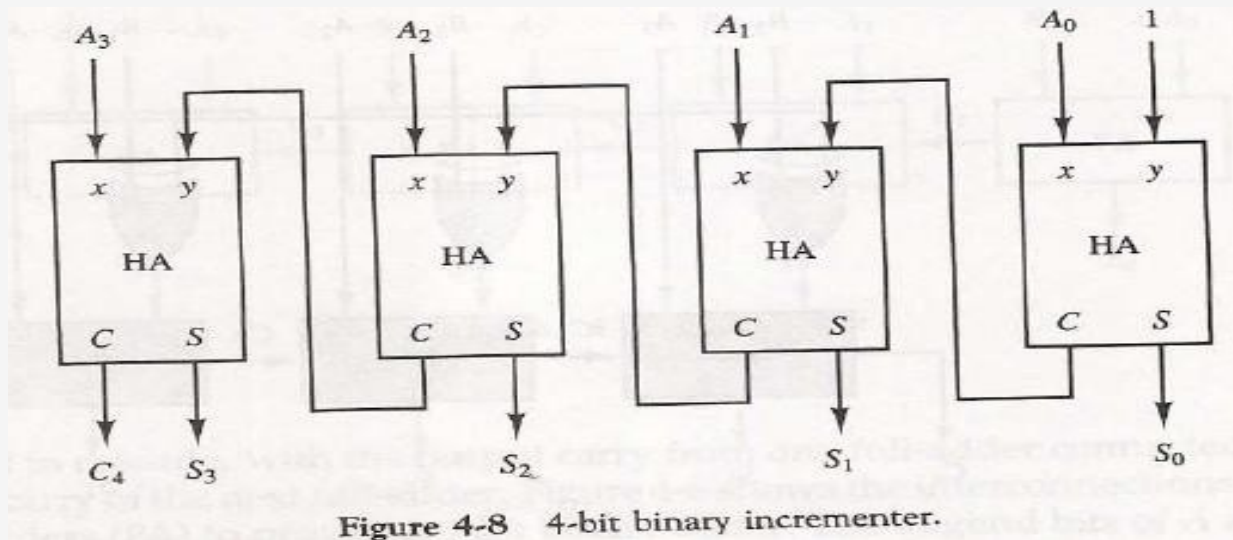


Figure 4-7 4-bit adder-subtractor.

Binary Incrementer:

- ❖ The increment microoperation adds one to a number in a register.
- ❖ For example, if a 4-bit register has a binary value 0110, it will go to 0111 after it is
- ❖ The least significant bit must have one input connected to logic-1. The other inputs receive the number to be incremented or the carry from the previous stage



Logic Micro-operations

- Logic microoperations specify binary operations for strings of bits stored in registers.
- These operations consider each bit of the register separately and treat them as binary variables.
- For example, the exclusive-OR microoperation with the contents of two registers R1 and R2 is symbolized by the statement

$$P: R1 \leftarrow R1 \oplus R2$$

List of Logic Microoperations:

- There are 16 different logic operations that can be performed with two binary variables.

Cont..

- The 16 Boolean functions of two variables x and y are expressed in algebraic form in the first column of Table

TABLE 4.5 Truth Tables for 16 Functions of Two Variables

[illegible]

Cont..

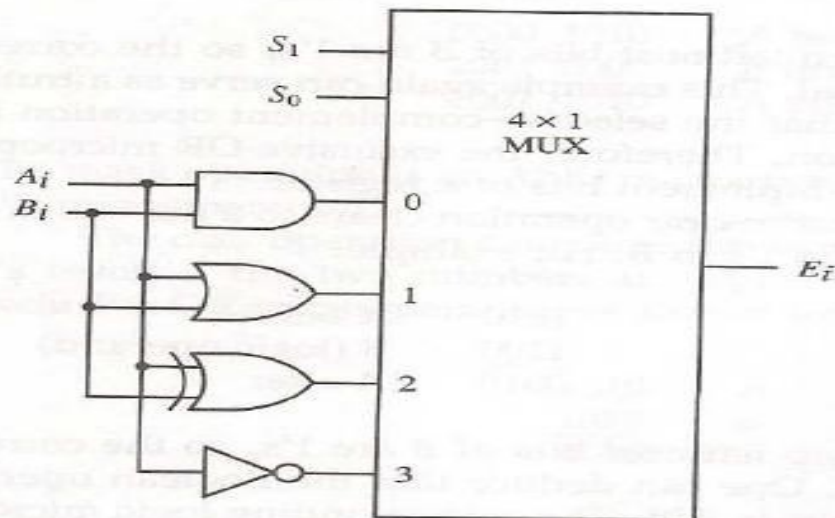
TABLE 4-6 Sixteen Logic Microoperations

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \overline{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \overline{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \overline{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \overline{B}$	
$F_{12} = x'$	$F \leftarrow \overline{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \overline{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

Hardware Implementation:

- The hardware implementation of logic microoperations requires that logic gates be inserted for each bit or pair of bits in the registers to perform the required logic function.
- Although there are 16 logic microoperations, most computers use only four--AND, OR, XOR (exclusive-OR), and complement from which all others can be derived.

Figure 4-10 One stage of logic circuit.



(a) Logic diagram

S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

(b) Function table

Some Applications

- Logic micro-operations are very useful for manipulating individual bits or a portion of a word stored in a register.
- They can be used to change bit values, delete a group of bits or insert new bits values into a register.

Selective set

- ✓ The *selective-set* operation sets to 1 the bits in register A where there are corresponding 1's in register B.
- ✓ The *OR microoperation* can be used to selectively set bits of a register.

1010

A before

1100

B (logic operand)

1110

A after

Cont..

Selective complement

- The selective-complement operation complements bits in A where there are corresponding 1's in B .
- The *exclusive-OR microoperation* can be used to selectively complement bits of a register.

1010	A before
<u>1100</u>	B (logic operand)
0110	A after

Selective clear

- The selective-clear operation clears to 0 the bits in A only where there are corresponding 1's in B.

For example:

1010	A before
<u>1100</u>	B (logic operand)
0010	A after

Cont..

➤ Mask

- ✓ The *mask* operation is similar to the selective-clear operation **except that the bits of A are cleared only where there are corresponding O's in B**.
- ✓ The mask operation is an AND micro operation as seen from the following numerical example:

0110 1010	A before
0000 1111	B (mask)
<u>0000 1010</u>	A after masking

➤ Insert

- ✓ The *insert* operation inserts a new value into a group of bits.
- ✓ This is done by first masking the bits and then ORing them with the required value.

Cont..

- ❖ For example, suppose that an A register contains eight bits, 0110 1010. To replace the four leftmost bits by the value 1001 we first mask the four unwanted bits:

0110 1010	A before
0000 1111	B (mask)
<u>0000 1010</u>	A after masking

and then insert the new value:

0000 1010	A before
1001 0000	B (insert)
<u>1001 1010</u>	A after insertion

Cont..

➤ Clear

- ✓ The *clear* operation compares the words in A and B and produces an all 0's result if the two numbers are equal.
- ✓ This operation is achieved by an *exclusive-OR microoperation* as shown by the following example

1010	A
1010	B
<u>1010</u>	
0000	$A \leftarrow A \oplus B$

Shift Microoperations:

- Shift microoperations are used for serial transfer of data.
- The contents of a register can be shifted to the left or the right.
- During a shift-left operation the serial input transfers a bit into the rightmost position.
- During a shift-right operation the serial input transfers a bit into the leftmost position.
- There are three types of shifts:
 - 1) logical,
 - 2) circular
 - 3) arithmetic.



Reading assignment

Thank you

?